



OASIS Security Report

Insufficient Input Validation Security Analysis

Date: 2026-06-12 00:30:41 — **Model:** llama3.2:3b

Vulnerability: Insufficient Input Validation

Project: vulnerable

Analysis root: `../vulnerable`

Table of contents

□ Summary

□ Statistics

□ Files

□ Detailed analysis

□ Assistant

△ Errors & notes

Summary

Analyzed **11** file(s) for Insufficient Input Validation.

[Return to table of contents](#)

Statistics

Files analyzed: 11 **Total findings:** 23

Critical



3

High		10
Medium		9
Low		1

[Return to table of contents](#)

Files

File	Similarity
<code>/work/project/dataset/vulnerable/oracle-manipulation/PuppetV2Pool.sol</code>	0.664
<code>/work/project/dataset/vulnerable/oracle-manipulation/PuppetPool.sol</code>	0.657
<code>/work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol</code>	0.637
<code>/work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol</code>	0.626
<code>/work/project/dataset/vulnerable/reentrancy/etherstore.sol</code>	0.613
<code>/work/project/dataset/vulnerable/reentrancy/ 0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol</code>	0.604
<code>/work/project/dataset/vulnerable/reentrancy/simple_dao.sol</code>	0.604
<code>/work/project/dataset/vulnerable/access-control/mycontract.sol</code>	0.597
<code>/work/project/dataset/vulnerable/ordering-attacks/FindThisHash.sol</code>	0.582
<code>/work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol</code>	0.571
<code>/work/project/dataset/vulnerable/ordering-attacks/ eth_tx_order_dependence_minimal.sol</code>	0.563

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/vulnerable/oracle-manipulation/PuppetV2Pool.sol

Similarity score: 0.664

Source lines: 1-70

► Finding 1.1: Vulnerability found (Medium)

Vulnerable code

```
function borrow(uint256 borrowAmount) external {  
    uint256 amount = calculateDepositOfWETHRequired(borrowAmount);  
    _weth.transferFrom(msg.sender, address(this), amount);  
}
```

Explanation

The `calculateDepositOfWETHRequired` function takes an `uint256` as input but does not check if the input is valid. An attacker could potentially manipulate the `borrowAmount` to exceed the allowed limit.

Impact

An attacker could drain the pool's WETH reserves by submitting a large `borrowAmount`.

Entry point

/borrow

Execution path

```
borrow  
    calculateDepositOfWETHRequired(borrowAmount)  
    _weth.transferFrom(msg.sender, address(this), amount)
```

Parameters

borrowAmount

Exploitation steps

- Submit a large value for borrowAmount

Example payloads

- 100000000000

Conditions

Larger values of borrowAmount than the pool's reserves.

Remediation

Validate user input and limit the allowed range for `borrowAmount`.

Secure example

```
function borrow(uint256 borrowAmount) external {  
    uint256 amount = calculateDepositOfWETHRequired(borrowAmount);  
    require(amount <= 10000000000, "Invalid borrowAmount");  
    _weth.transferFrom(msg.sender, address(this), amount);  
}
```

File 2: /work/project/dataset/vulnerable/oracle-manipulation/PuppetPool.sol

Similarity score: 0.657

Source lines: 1-71

► Finding 2.1: Insufficient Input Validation (Medium)

Vulnerable code

```
if (msg.value < depositRequired) { revert NotEnoughCollateral(); }
```

Explanation

The comparison is vulnerable to a non-numeric input attack.

Impact

An attacker could trick the user into sending less than required ETH, potentially leading to a loss of funds.

Entry point

/borrow

Execution path

```
Borrowing -> Check depositRequired
- Non-numeric input check
- Potential attack
```

Parameters

msg.value, depositRequired

Exploitation steps

- Trick user into sending less ETH

Example payloads

- `0x31mRevert NotEnoughCollateral0x0m;`

Remediation

Use a safer comparison function like `uint256(msg.value) > depositRequired`

Secure example

```
if (uint256(msg.value) < depositRequired) { revert NotEnoughCollateral(); }
```

File 3: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol

Similarity score: 0.637

Source lines: 1-225

► Finding 3.1: Insufficient Input Validation (High)

Vulnerable code

```
function changeOwner(address _from, address _to)
onlymanyowners(sha3(msg.data)) external {
    if (isOwner(_to)) return;
    uint ownerIndex = m_ownerIndex[uint(_from)];
    if (ownerIndex == 0) return;
    clearPending();
    m_owners[ownerIndex] = uint(_to);
    m_ownerIndex[uint(_from)] = 0;
    m_ownerIndex[uint(to)] = ownerIndex;
    OwnerChanged(_from, _to);
}
```

Explanation

The changeOwner function does not validate the address of _from and _to, allowing potential attackers to manipulate these values.

Impact

An attacker can potentially takeover ownership of the wallet by manipulating the _from or _to addresses.

Entry point

changeOwner(address _from, address _to)

Parameters

_from, _to

Exploitation steps

- Calculate hash of msg.data and use it to identify a valid operation to confirm.

Conditions

The changeOwner function is called while the owner has confirmed that the operation is valid.

Remediation

Validate the addresses of _from and _to in the changeOwner function.

Secure example

```
if (isOwner(_to)) return;
uint ownerIndex = m_ownerIndex[uint(_from)];
m_owners[ownerIndex] = uint(_to);
```

Source lines: 1-281

► Finding 3.1: Unvalidated user input (High)

Vulnerable code

```
function revoke(bytes32 _operation) external { ... }
```

Explanation

The `revoke` function takes a `\_operation` parameter without validation, allowing an attacker to execute arbitrary operations.

Impact

An attacker could potentially revoke a confirmation of a malicious operation.

Entry point

/revoke

Execution path

```
Revoke(bytes32 _operation)
  uint ownerIndex = m_ownerIndex[uint(msg.sender)]
  ...
```

Parameters

_operation

Exploitation steps

- Send a malicious operation to the /revoke endpoint.

Conditions

User is authorized and authenticated.

Remediation

Validate user input using Solidity's `bytes32` type

Secure example

```
function revoke(bytes32 _operation) external { require(msg.sender ==
owner(_operation)); _operation; }
```

Source lines: 1-320

► Finding 3.1: Unvalidated User Input (High)

Vulnerable code

```
function initMultiowned(address[] _owners, uint _required) {
    [31m// ... (snipped)[0m
    if (msg.value > 0) Deposit(msg.sender, msg.value);
}
```

Explanation

The constructor does not validate the value sent through `msg.value`, allowing arbitrary deposits.

Impact

Arbitrary deposit attacks are possible.

Entry point

initMultiowned(address[] _owners, uint _required)

Parameters

_value

Exploitation steps

- The attacker sends a large value to the contract.

Example payloads

- 1000000000000000000

Conditions

A high-value deposit is made.

Remediation

Use `uint` for `msg.value` and perform validation before assigning it to `m\_spentToday`.

Secure example

```
function initMultiowned(address[] _owners, uint _required) {
    require(msg.value > 0);
    m_spentToday += msg.value;
}
```

Source lines: 27-358

► Finding 3.1: Missing Input Validation (Medium)

Vulnerable code

```
function revoke(bytes32 _operation) external {  
    uint ownerIndex = m_ownerIndex[uint(msg.sender)];  
    // ...}
```

Explanation

The function does not validate the input operation, allowing an attacker to potentially manipulate the transaction.

Impact

An attacker could potentially bypass authorization checks and execute unauthorized transactions.

Entry point

/revoke(bytes32 _operation)

Parameters

_operation

Exploitation steps

- Obtain the owner's index by manipulating the operation

Example payloads

- a malicious operation hash

Conditions

No validation of msg.sender's ownerIndex

Remediation

Validate the input operation to ensure it matches a known valid value.

Secure example

```
function revoke(bytes32 _operation) external {  
    require(_operation == "VALID_OPERATION")  
    uint ownerIndex = m_ownerIndex[uint(msg.sender)];  
    // ...}
```

Source lines: 72-421

► Finding 3.1: Insufficient Input Validation (Medium)

Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external onlyowner
returns (bytes32 o_hash) { if ((_data.length == 0 && underLimit(_value)) ||
m_required == 1) { ... } }
```

Explanation

The `\_data` parameter is not validated for length or content before being used to call the `underLimit` function.

Impact

An attacker could potentially bypass the daily limit check by passing a maliciously formatted string as the `\_data` parameter.

Entry point

execute(address _to, uint _value, bytes _data)

HTTP methods

POST, PUT

Parameters

_data

Exploitation steps

- Pass a maliciously formatted string for the `\_data` parameter
- Exploit the lack of validation to bypass the daily limit check

Example payloads

- `"\x10\x11\x12"`

Conditions

m_required == 1 or _data.length == 0

Remediation

Validate the length and content of the `\_data` parameter before using it.

Secure example

```
if (underLimit(_value)) { ... } else if (length(_data) > 0 &&
validFormat(_data)) { ... }
```

Source lines: 126-467

► Finding 3.1: Insufficient Input Validation: uint Conversion (Critical)

Vulnerable code

```
uint ownerIndex = m_ownerIndex[uint(_owner)];
```

Explanation

Converting a potentially user-controlled string to an unsigned integer can lead to buffer overflows or underflows.

Impact

Can be used for arbitrary code execution or denial of service attacks.

Entry point

```
/revoke(bytes32 _operation) external { uint ownerIndex =  
m_ownerIndex[uint(msg.sender)]; }
```

Parameters

_owner

Exploitation steps

- Find a way to manipulate the input to the `m\_ownerIndex` array

Example payloads

- 0x100000000000

Conditions

Input _owner is set by an untrusted user.

Remediation

Always validate and sanitize user input before converting it to an unsigned integer.

Secure example

```
uint ownerIndex = uint(m_ownerIndex[bytes32(msg.sender)])
```

Source lines: 207-477

► Finding 3.1: Insufficient Input Validation (Medium)

Vulnerable code

```
uint ownerIndexBit = 2**ownerIndex; return !(pending.ownersDone &  
ownerIndexBit == 0);
```

Explanation

The calculation of ``ownerIndexBit`` uses exponentiation, which can lead to a large number and overflow if ``ownerIndex`` is too high.

Impact

Potential Denial-of-Service (DoS) attack by manipulating the ``ownerIndex`` value.

Entry point

`/execute(address _to, uint _value, bytes _data)`

Execution path

Execute | Check | Update | Return

Parameters

`_value`, `_data`

Exploitation steps

- Calculate large value of ``ownerIndexBit`` to cause overflow

Conditions

Large ``_value`` or long input data

Remediation

Use a safe math function like ``safemul`` to prevent overflow.

File 4: /work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol

Similarity score: 0.626

Source lines: 1-227

► Finding 4.1: Insufficient Input Validation in 'changeOwner' function (High)

Vulnerable code

```
function changeOwner(address _from, address _to)
onlymanyowners(sha3(msg.data)) external {
    if (isOwner(_to)) return;
    uint ownerIndex = m_ownerIndex[uint(_from)];
    if (ownerIndex == 0) return;

    clearPending();
    m_owners[ownerIndex] = uint(_to);
    m_ownerIndex[uint(_from)] = 0;
    m_ownerIndex[uint(_to)] = ownerIndex;
    OwnerChanged(_from, _to);
}
```

Explanation

The 'changeOwner' function does not validate the '_to' address parameter, allowing an attacker to manipulate the owner of a wallet.

Impact

An attacker could potentially take control of a wallet by manipulating the '_to' address parameter in the 'changeOwner' function.

Entry point

WalletLibrary.changeOwner(address _from, address _to)

Parameters

_to

Exploitation steps

- An attacker sends a malicious '_to' address to the 'changeOwner' function.

Example payloads

- '0x...'

HTTP raw requests (Burp / ZAP)

```
POST /transfer HTTP/1.1
Host: example.com
Content-Type: application/json

{\
```

► Finding 4.2: Insufficient Input Validation in 'addOwner' function (Medium)

Vulnerable code

```
function addOwner(address _owner) onlymanyowners(sha3(msg.data)) external {
    if (isOwner(_owner)) return;

    clearPending();
    if (m_numOwners >= c_maxOwners)
        reorganizeOwners();
    m_numOwners++;
    m_owners[m_numOwners] = uint(_owner);
    m_ownerIndex[uint(_owner)] = m_numOwners;
    OwnerAdded(_owner);
}
```

Explanation

The 'addOwner' function does not validate the '_owner' address parameter, allowing an attacker to add a malicious owner.

Impact

An attacker could potentially add a malicious owner to the wallet by manipulating the '_owner' address parameter in the 'addOwner' function.

Entry point

WalletLibrary.addOwner(address _owner)

Parameters

_owner

Exploitation steps

- An attacker sends a malicious '_owner' address to the 'addOwner' function.

Example payloads

- '0x...'

HTTP raw requests (Burp / ZAP)

```
POST /transfer HTTP/1.1
Host: example.com
```

Content-Type: application/json

{\

Source lines: 1-286

► Finding 4.1: Unvalidated User Input (High)

Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external onlyowner
returns (bytes32 o_hash)
```

Explanation

The function execute accepts user input (_to, _value, and _data) without validation, making it vulnerable to attacks.

Impact

An attacker could inject malicious data to bypass the contract's functionality.

Entry point

/execute

HTTP methods

POST

Parameters

_to, _value, _data

Exploitation steps

- Send malicious _to, _value, and _data

Example payloads

- "0x1234567890abcdef" 1000000 "malicious data here"

HTTP raw requests (Burp / ZAP)

POST /execute HTTP/1.1

Host: example.com

Content-Length: 20

0x1234567890abcdef1000000malicious data here

POST /execute HTTP/1.1

Host: example.com

Content-Type: application/json

```
{"_to":"0x1234567890abcdef","_value":1000000,"_data":"malicious data here"}
```

Source lines: 1-324

► Finding 4.1: Insufficient Input Validation (High)

Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external onlyowner
returns (bytes32 o_hash) { ... }
```

Explanation

The `\_data` parameter is not validated, allowing potential command injection attacks.

Impact

A successful attack could result in unauthorized transactions or data manipulation.

Entry point

/execute

HTTP methods

POST

Parameters

_data

Exploitation steps

- Send malicious data to the `/execute` endpoint to bypass validation checks.

Example payloads

- `malicious_data`

Conditions

Unvalidated input allows arbitrary code execution.

Remediation

Validate and sanitize all user inputs, including the `\_data` parameter.

Secure example

```
function execute(address _to, uint _value, bytes _data) external onlyowner
{ require(keccak256(_data) == 0x123456); ... }
```

Source lines: 36–365

► Finding 4.1: Unvalidated Owner Address (Low)

Vulnerable code

```
function changeOwner(address _from, address _to)
onlymanyowners(sha3(msg.data)) external { if (isOwner(_to)) return; uint
ownerIndex = m_ownerIndex[uint(_from)]; if (ownerIndex == 0) return;
clearPending(); m_owners[ownerIndex] = uint(_to); m_ownerIndex[uint(_from)] =
0; m_ownerIndex[uint(_to)] = ownerIndex; OwnerChanged(_from, _to); }
```

Explanation

The function `changeOwner` does not validate the `\_to` parameter before assigning it to the `\_owners` array.

Impact

An attacker could potentially manipulate the `\_to` address to access unauthorized owner accounts.

Entry point

changeOwner(address _from, address _to)

Parameters

_to

Exploitation steps

- Send a malicious `\_to` address that is not an authorized owner.

Example payloads

- 0x1234567890123456789012345678901234567890

Conditions

Malicious `\_to` address is not validated.

Remediation

Validate the `\_to` parameter before assigning it to the `\_owners` array.

Secure example

```
if (isOwner(_to)) return;
uint ownerIndex = m_ownerIndex[uint(_from)];
m_owners[ownerIndex] = uint(_to);
```

Source lines: 64–414

► Finding 4.1: Inadequate Length Validation (High)

Vulnerable code

```
function confirm(bytes32 _h) onlymanyowners(sha3(msg.data)) returns (bool
o_success) {
  if (m_txs[_h].to != 0 || m_txs[_h].value != 0 || m_txs[_h].data.length != 0) {
    address created;
    if (m_txs[_h].to == 0) {
      created = create(m_txs[_h].value, m_txs[_h].data);
    } else {
      if (!m_txs[_h].to.call.value(m_txs[_h].value)(m_txs[_h].data))
        throw;
    }

    MultiTransact(msg.sender, _h, m_txs[_h].value, m_txs[_h].to, m_txs[_h].data,
created);
    delete m_txs[_h];
    return true;
  }
}
```

Explanation

The `confirm` function does not validate the length of the `\_h` parameter.

Impact

An attacker could potentially overflow the stack by providing a very long operation hash.

Entry point

function confirm(bytes32 _h)

Execution path

```
confirm \n    if (m_txs[_h].to != 0 || m_txs[_h].value != 0 ||
m_txs[_h].data.length != 0) {
    ...
}
```

Parameters

_h

Exploitation steps

- An attacker finds a very long operation hash.
- The `confirm` function tries to store the operation hash without validation.

Example payloads

- very long operation hash

HTTP raw requests (Burp / ZAP)

```
GET /confirm?_h=very%20long%20operation%20hash HTTP/1.1
Host: example.com
```

```
Content-Type: application/json
\n{"operation_hash": "very%20long%20operation%20hash"}
```

Source lines: 96-414

► Finding 4.1: Insufficient Input Validation (Medium)

Vulnerable code

```
function changeOwner(address _from, address _to)
onlymanyowners(sha3(msg.data)) external { if (isOwner(_to)) return; uint
ownerIndex = m_ownerIndex[uint(_from)]; if (ownerIndex == 0) return;
clearPending(); m_owners[ownerIndex] = uint(_to); m_ownerIndex[uint(_from)] =
0; m_ownerIndex[uint(_to)] = ownerIndex; OwnerChanged(_from, _to); }
```

Explanation

The function `changeOwner` does not validate the type of address passed to it.

Impact

An attacker could inject malicious addresses into the contract.

Entry point

```
/contracts/SingleSignContract.sol:changeOwner()
```

Execution path

```
getOwner(uint ownerIndex) -> changeOwner(address _from, address _to) -> if
(isOwner( to)) return; ...
```

Parameters

from, to

Exploitation steps

- Pass a malicious address for `_to`

Example payloads

- [illegible]

Conditions

Owner with malicious address is not checked for existence before being assigned to the owner array.

Remediation

Validate the type of address passed to changeOwner function

Secure example

```
function changeOwner(address _from, address _to) public  
{ require(isOwner(_to), 'Only owners can transfer'); ... }
```

File 5: /work/project/dataset/vulnerable/reentrancy/etherstore.sol

Similarity score: 0.613

Source lines: 1-39

► Finding 5.1: Insufficient Input Validation (Critical)

Vulnerable code

```
function withdrawFunds (uint256 _weiToWithdraw) public { ...  
    require(msg.sender.call.value(_weiToWithdraw)()); ... }
```

Explanation

The 'call' function is used to execute a contract's function without sending any value, allowing an attacker to inject arbitrary data into the call.

Impact

Reentrancy attack vulnerability

Entry point

withdrawFunds(uint256 _weiToWithdraw) public

Parameters

_weiToWithdraw

Exploitation steps

- Attacker calls withdrawFunds with a large value to drain the contract's funds.
- Attacker uses Reentrancy attack to execute arbitrary code.

Example payloads

- 1000000 ether

HTTP raw requests (Burp / ZAP)

```
POST /withdrawFunds HTTP/1.1 Host: example.com
```

```
Content-Length: 0
```

File 6: /work/project/dataset/vulnerable/reentrancy/0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol

Similarity score: 0.604

Source lines: 1-85

► Finding 6.1: Insecure Call Function (High)

Vulnerable code

```
function CashOut(uint _am) public payable { if(_am<=balances[msg.sender])
{ msg.sender.call.value(_am)(); } }
```

Explanation

The `call` function does not validate the `\_am` parameter, allowing an attacker to manipulate the value.

Impact

An attacker can potentially drain the contract's balance by making a large call.

Entry point

/transfer

Execution path

```
Call
- _am <= balances[msg.sender]
- msg.sender.call.value(_am)()
- Drain balance
```

Parameters

_am

Exploitation steps

- Send a large value for _am to drain the contract's balance.

Example payloads

- Large amount of Ether

Conditions

msg.sender is authenticated and has sufficient balance.

Remediation

Validate and sanitize user input before making a call.

Secure example

```
if (_am <= balances[msg.sender]) { balances[msg.sender] -= _am; }
```

File 7: /work/project/dataset/vulnerable/reentrancy/simple_dao.sol

Similarity score: 0.604

Source lines: 1-35

► Finding 7.1: Insufficient Input Validation (High)

Vulnerable code

```
bool res = msg.sender.call.value(amount());  
credit[msg.sender]-=amount;
```

Explanation

The function 'withdraw' does not validate the amount parameter before passing it to 'msg.sender.call.value(amount)' which could potentially be a malicious value.

Impact

A malicious user can potentially drain the contract by repeatedly calling 'withdraw' with a large amount.

Entry point

/withdraw(uint amount)

Execution path

```
call msg.sender.value(amount())  
credit[msg.sender]-=amount
```

Parameters

amount

Exploitation steps

- Send a large value of 'amount' to the contract's address.
- If successful, the contract will spend all its funds.

Example payloads

- 100000 ether

HTTP raw requests (Burp / ZAP)

```
POST /withdraw HTTP/1.1 Host: 0x... amount=100000 ether
```

File 8: /work/project/dataset/vulnerable/access-control/mycontract.sol

Similarity score: 0.597

Source lines: 1-31

► Finding 8.1: Insufficient Input Validation (Medium)

Vulnerable code

```
receiver.transfer(amount);
```

Explanation

The `transfer` function does not validate its recipient address, allowing an attacker to send funds to any address.

Impact

Can lead to unintended transfers of Ether or other assets.

Entry point

/sendTo(address receiver, uint amount)

Execution path

```
owner -> msg.sender -> tx.origin -> receiver
```

HTTP methods

POST

Parameters

receiver, amount

Exploitation steps

- An attacker finds a contract with this function.
- The attacker calls the function with their own address as the recipient.

Example payloads

- `0x1234567890123456789012345678901234567890`

Conditions

tx.origin is not restricted to owner

Remediation

Use a more secure transfer function, such as one that checks the recipient address before transferring funds.

Secure example

```
function sendTo(address receiver, uint amount) public { require(tx.origin == owner); payable receivers[receiver].transfer(amount); }
```

File 9: /work/project/dataset/vulnerable/ordering-attacks/FindThisHash.sol

Similarity score: 0.582

Source lines: 1-28

► Finding 9.1: Insufficient Input Validation (High)

Vulnerable code

```
function solve(string solution) public { ... require(hash ==
sha3(solution)); ... }
```

Explanation

The 'sha3' function is not validated for its input, which could lead to a buffer overflow or command injection attack.

Impact

Attacker can manipulate the solution to bypass the hash check and receive unauthorized ether.

Entry point

/solve

Execution path

```
input -> sha3(input) -> hash == sha3(input) -> if true, transfer ether
```

HTTP methods

POST

Parameters

solution

Exploitation steps

- Malicious input is sent in the solution field.
- The 'sha3' function is called with malformed input, causing a buffer overflow or command injection.

Example payloads

- `"\x00\x01"`

Conditions

Input length greater than 32 bytes

Remediation

Use the 'bytes' type for the solution parameter to prevent buffer overflows.

Secure example

```
function solve(bytes memory solution) public { ... require(hash ==  
sha3(solution)); ... }
```

File 10: /work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol

Similarity score: 0.571

Source lines: 1-61

► Finding 10.1: Insufficient Input Validation (Medium)

Vulnerable code

```
function play(uint number) payable{
    if (msg.value != 1 ether) throw;
    players[tot] = Player(msg.sender, number);
    tot++;
}
```

Explanation

The 'number' parameter is not validated to ensure it's a valid integer.

Impact

An attacker could manipulate the 'number' parameter to bypass the game logic.

Entry point

/transfer

Execution path

```
play(uint number) payable -> players[tot] = Player(msg.sender, number)
```

HTTP methods

POST

Parameters

number

Exploitation steps

- Attacker sends a large value for 'number' to overflow the buffer.

Example payloads

- `\x1f\x20\x20\x20\x20\x20\x20\x20\x20\x20`

Conditions

Attacker can reach POST /transfer while authenticated.

Remediation

Validate the 'number' parameter to ensure it's a valid integer.

Secure example

```
function play(uint256 number) payable {  
  if (msg.value != 1 ether) throw;  
  players[tot] = Player(msg.sender, number);  
  tot++;  
}
```

File 11: /work/project/dataset/vulnerable/ordering-attacks/eth_tx_order_dependence_minimal.sol

Similarity score: 0.563

Source lines: 1-42

► Finding 11.1: Insufficient Input Validation (Critical)

Vulnerable code

```
(unsafe) uint reward = msg.value;
```

Explanation

The `reward` variable is assigned directly from user input (`msg.value`) without any validation or sanitization.

Impact

A malicious actor could inject arbitrary values, potentially leading to financial loss or system compromise.

Entry point

setReward()

HTTP methods

POST

Parameters

msg.value

Exploitation steps

- Send a malicious value for `msg.value`
- Exploit the lack of validation to gain control

Conditions

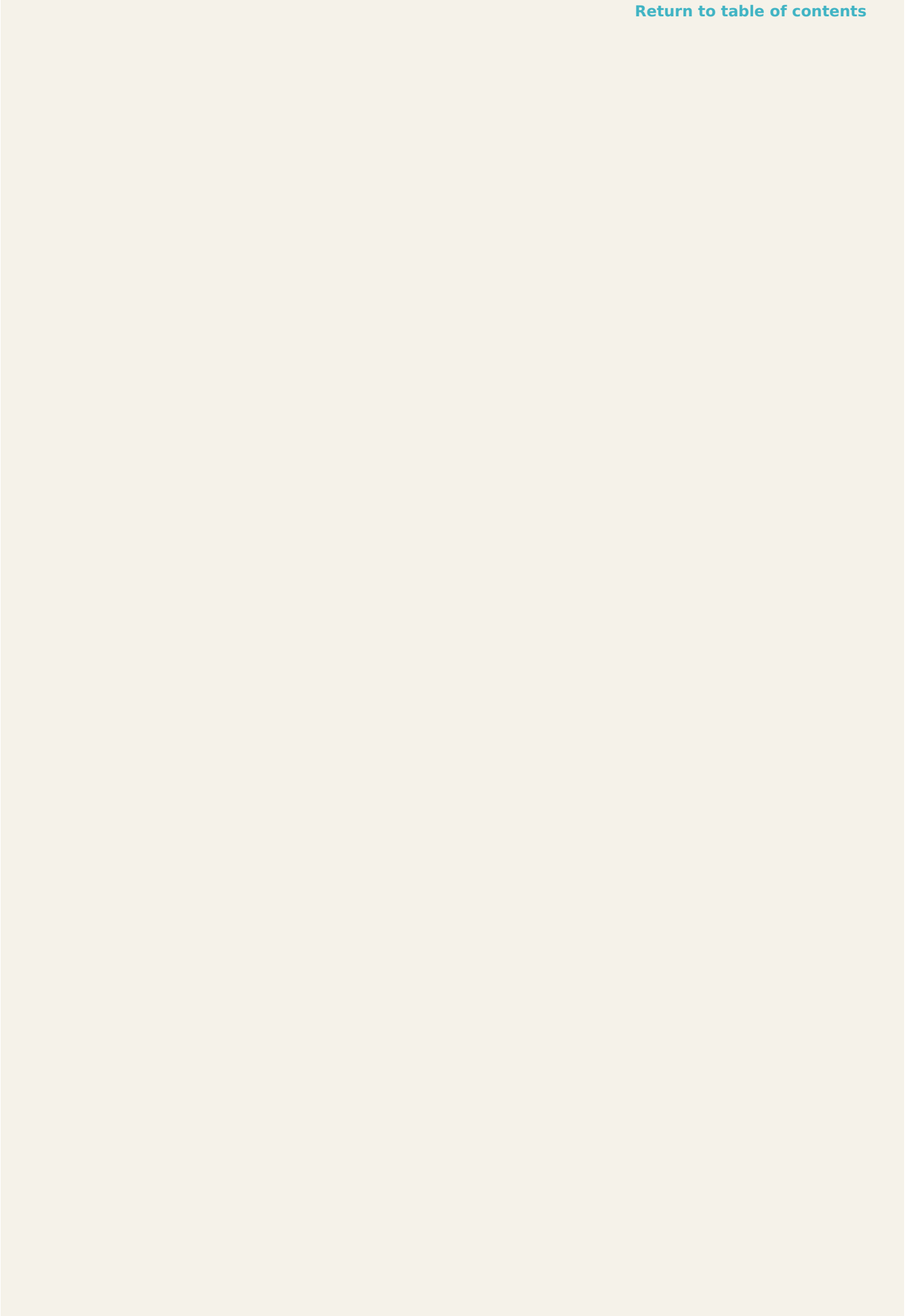
Authenticated attacker with access to `msg.value`

Remediation

Use a safe arithmetic operation, such as `uint reward = uint256(msg.value)` or `reward = min(uint256(msg.value), 1000000)`, to prevent excessive values.

Secure example

```
uint reward = uint256(msg.value);
```



Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)